

Swiss Post Package Sorting Center

RL Optimization v1.6 — Deep Step-by-Step Explanation

Author: Moataz Mansour

Course: CAS Advanced Machine Learning 2026 — Module 6

Date: June 2026

Scope: Every part, cell, and decision explained in depth

This document provides a comprehensive, step-by-step walkthrough of the entire Swiss Post RL project. Every part of the notebook is explained: what it does, why it was designed that way, how the code works line by line, and what the results mean. It covers the theory, the data pipeline, all three environment versions, every from-scratch algorithm implementation, the SB3 training, evaluation methodology, visualizations, and business conclusions.

Table of Contents

- Part A: Project Overview & Why Reinforcement Learning
- Part B: Data Loading, Cleaning & Feature Engineering (Cells 3–9)
- Part C: Environment V1 — First MDP Implementation (Cells 10–18)
- Part D: From-Scratch Q-Learning Agent (Cells 19–21)
- Part E: From-Scratch SARSA Agent (Cells 22–24)
- Part F: Environment V2 — Harder, More Realistic (Cells 25–26)
- Part G: Environment V3 — Action Masking + Curriculum (Cells 27–28)
- Part H: From-Scratch DQN in PyTorch (Cells 29–31)
- Part I: What is SB3? — Stable-Baselines3 Explained
- Part I-2: The 4 SB3 Models Explained (DQN, A2C, PPO, MaskablePPO)
- Part I-3: A2C Deep Dive — Advantage Actor-Critic
- Part I-3b: PPO Deep Dive — Proximal Policy Optimization
- Part I-4: SB3 Training with Curriculum Learning (Cells 32–33)
- Part J: Temporal Train/Test Split & Evaluation (Cells 34–37)
- Part J-2: Results Dashboard — 10-Policy Comparison (Visual Analysis)
- Part K: Cost-Benefit Analysis (Cells 38–39)
- Part L: Policy Heatmap Analysis (Cells 40–41)
- Part M: Hyperparameter Grid Search (Cells 42–43)
- Part N: Multi-Run Statistical Evaluation (Cells 44–45)

- Part O: Model Persistence — Save & Load (Cells 46–47)
- Part P: Visual Simulation Demo (Cells 48–55)
- Part Q: Interactive Live Animation (Cells 56–61)
- Part R: Conclusions & Business Recommendations (Cell 62)

Part A: Project Overview & Why Reinforcement Learning

The Problem

Swiss Post's package sorting centers process thousands of packages per hour across multiple sorting lanes (called **chutes**). During peak hours, chutes become congested — queues build up, processing times spike, and delays cascade through the entire system. The fundamental question is: can a machine learning agent learn *when* and *how* to intervene (add or remove capacity, reroute packages, apply priority processing) to minimize delays while keeping operational costs low?

Why Not Supervised Learning?

A supervised learning model would need labeled training data of the form (situation → best action). But we do not have this. We only have historical data showing what *happened*, not what the *optimal action would have been*. Furthermore, sorting center optimization is a **sequential decision problem**: the action you take now affects the state you will see in the next hour. Adding a chute now costs money but prevents a queue explosion later. Supervised learning treats each decision independently and cannot reason about these temporal tradeoffs.

Approach	Strength	Limitation
Rule-based	Simple, interpretable	Cannot adapt to unseen patterns
LSTM Forecasting	Predicts future load	Descriptive, not prescriptive — tells you what will happen, not what to do
Reinforcement Learning	Learns optimal actions by trial and error	Needs careful environment design

Why Reinforcement Learning Fits

RL is designed for exactly this type of problem: an agent interacts with an environment, takes actions, observes rewards, and learns a policy that maximizes long-term cumulative reward. The agent can explore "what if" scenarios in simulation (what if I add a chute before the peak hour?) without needing labeled data. The reward function lets us encode complex business tradeoffs (delay penalty vs. chute cost) naturally.

The MDP Framework

Every RL problem is formalized as a **Markov Decision Process (MDP)**, defined by the tuple $\langle S, A, P, R, \gamma \rangle$ where S is the state space, A is the action space, $P(s'|s,a)$ is the transition function, $R(s,a)$ is the reward function, and γ is the discount factor. The "Markov" property means the next state depends only on the current state and action, not on the full history. This is a reasonable assumption for our sorting center: knowing the current queue length, load, and number of active chutes is sufficient to make a good decision without knowing the entire history.

Course Alignment

This project implements algorithms from every major lecture in the CAS AML Module 6 curriculum:

Algorithm	Course Lecture	Implementation
Q-Learning	Lecture 5 (Taxi)	From-scratch tabular agent
SARSA	Lecture 5 (MountainCar)	From-scratch on-policy agent
DQN	Lecture 10 (CartPole)	From-scratch PyTorch + SB3
A2C	Lecture 11	SB3 Advantage Actor-Critic

Algorithm	Course Lecture	Implementation
PPO	Lecture 13	SB3 PPO + MaskablePPO
MDP Theory	Lecture 2	Explicit formulation in all environments

Part B: Data Loading, Cleaning & Feature Engineering

Notebook Cells: 3–9

Step 1: Load the Raw CSV

The dataset `data4day_with_issues.csv` contains 4 days of hourly package sorting data from a real Swiss Post sorting center. Each row represents one combination of hour × chute × ZIP code, with the number of packages and average processing time.

```
df = pd.read_csv('data4day_with_issues.csv') df['HOUR_TIME'] = pd.to_datetime(df['HOUR_TIME'])
df['HOUR'] = df['HOUR_TIME'].dt.hour
```

The `HOUR_TIME` column is parsed from string to datetime so we can extract the hour of day (0–23) for each record. This hour will become a key feature in the RL state vector — it tells the agent what time of day it is, which directly correlates with demand patterns.

Step 2: Clean Negative Processing Times

The dataset contains 691 rows with negative `AVG_PROCESSING_TIME_MINUTES` — these are data quality issues (likely timestamp errors or recording glitches). They are removed rather than imputed, since they represent less than 0.5% of the data and negative processing times are physically meaningless.

```
n_negative = (df['AVG_PROCESSING_TIME_MINUTES'] < 0).sum() df = df[df['AVG_PROCESSING_TIME_MINUTES'] >= 0]
```

Step 3: Aggregate to Chute-Level Statistics

The raw data has one row per (hour, chute, ZIP code) combination. We aggregate to one row per (chute, hour) by summing packages, counting distinct ZIP codes, and averaging processing time:

```
summary = df.groupby(['CHUTE', 'HOUR']).agg( CHUTE_LOAD=('PACKAGE_COUNT', 'sum'), ZIP_COUNT=('ZIP_CODE', 'nunique'),
AVG_DELAY=('AVG_PROCESSING_TIME_MINUTES', 'mean') ).reset_index()
```

CHUTE_LOAD = total packages per chute per hour (the demand signal). **ZIP_COUNT** = number of distinct ZIP codes handled (a complexity indicator — more ZIPs means more diverse routing). **AVG_DELAY** = mean processing time across all packages in that hour/chute.

Step 4: Normalize Load to [0, 2] Range

```
max_load = summary['CHUTE_LOAD'].max() summary['CHUTE_LOAD_NORM'] = summary['CHUTE_LOAD'] / max_load * 2
```

The load is normalized to a [0, 2] range where 1.0 represents 50% of the maximum observed load and values above 1.0 indicate overload conditions. This normalized load becomes the primary demand signal in the RL environment. The factor of 2 (rather than 1) allows the environment to represent overload states where load exceeds the "normal" capacity.

Step 5: Build `sim_states` for the RL Environment

Each aggregated row becomes a simulation state dictionary that the RL environment will use to drive demand:

```
sim_states = [] for _, row in summary.iterrows(): sim_states.append( { 'chute': row['CHUTE'], 'hour':
int(row['HOUR']), 'load': float(row['CHUTE_LOAD_NORM']), 'zip_count': int(row['ZIP_COUNT']),
'avg_delay': float(row['AVG_DELAY']) })
```

This produces **6,694 simulation states** derived from real data. During training, the RL environment steps through these states sequentially (wrapping around), so the agent experiences realistic demand patterns — quiet early mornings, busy mid-mornings, afternoon peaks, and quiet evenings.

Key Insight: The `sim_states` bridge the gap between raw data and the RL environment. We cannot train RL directly on CSV rows because RL needs to take actions and observe consequences. The `sim_states` provide the demand pattern; the environment simulates what happens when the agent acts.

Step 6: Operational Activity Dashboard (Cell 8)

Before applying RL, the notebook visualizes the real data to understand the patterns the agent must learn. The dashboard includes: hourly package volume per chute (showing peak hours at 9–12 and 15–17), load distribution across chutes (identifying bottleneck chutes), processing time distributions (showing bimodal patterns — fast normal processing vs. slow congested processing), and a heatmap of load × hour showing when and where congestion occurs.

Part C: Environment V1 — First MDP Implementation

Notebook Cells: 10–18

The V1 environment (`PackagingCenterEnv`) is the first concrete implementation of the MDP. It is deliberately simple — the goal is to verify that the RL pipeline works end-to-end before adding complexity.

State Space (6 dimensions)

#	Feature	Range	What It Tells the Agent
1	chute_load	[0, 2]	How overloaded are the chutes? >1.0 = overload
2	hour_of_day	[0, 23]	What time is it? Determines demand pattern
3	day_of_week	[0, 6]	Weekday vs weekend — affects package volume
4	active_chutes	[1, max_chutes]	How many sorting lanes are currently active
5	queue_length	[0, 100]	Number of packages waiting to be sorted
6	processing_rate	[0.1, 2.0]	Current speed of package processing

These 6 numbers form the **observation vector** that the agent receives at each timestep. The agent must learn to map this 6-dimensional vector to the best action. The observation space is defined as a continuous Box in Gymnasium:

```
self.observation_space = spaces.Box( low=np.array([0.0, 0.0, 0.0, 1.0, 0.0, 0.1]), high=np.array([2.0, 23.0, 6.0, float(max_chutes), 100.0, 2.0]), dtype=np.float32 )
```

Action Space (5 discrete actions)

ID	Action	Effect	Cost	When to Use
0	Do Nothing	No intervention	Free	System is healthy — save money
1	Add Chute	+1 active chute	High (5.0)	Load is rising, need more capacity
2	Reroute	Reduce queue 20–40%	Medium (2.0)	Queue is building up dangerously
3	Priority Processing	Boost rate +30%	Medium (3.0)	Need to clear a backlog fast
4	Remove Chute	–1 active chute	Free	Load is low, save resources

The `step()` Function — How One Timestep Works

Each call to `env.step(action)` simulates one time interval. Here is the sequence:

- **1. Read demand from `sim_states`:** The environment reads the current `sim_state` to get the real-data-driven demand level for this hour.
- **2. Generate incoming packages:** A Poisson-distributed random number of packages arrive, scaled by the demand level and time of day.
- **3. Apply the agent's action:** If the agent chose Add Chute, `active_chutes` increases by 1. If Reroute, queue is reduced by 20–40%. If Priority, `processing_rate` increases by 30%. If Remove Chute, `active_chutes` decreases by 1.
- **4. Process packages:** $\text{capacity} = \text{active_chutes} \times \text{processing_rate} \times \text{base_capacity}$. The number of processed packages is $\min(\text{queue} + \text{incoming}, \text{capacity})$. Unprocessed packages stay in the queue.
- **5. Update state:** $\text{chute_load} = \text{incoming} / \text{capacity}$. Queue grows by $(\text{incoming} - \text{processed})$. Processing rate may decay slightly.
- **6. Compute reward:** A multi-factor reward function balances competing objectives (see below).

- **7. Check termination:** The episode ends after max_steps (100 in V1, 200 in V2/V3).

The Reward Function — The Steering Wheel

The reward function is the **most critical design decision** in any RL project. It tells the agent what "good" looks like. A poorly designed reward leads to degenerate policies (e.g., always add chutes regardless of need).

```
reward = - load_penalty # higher load = worse + throughput_bonus # processing more packages = good -  
overload_penalty # load > 1.0 = quadratic penalty - action_cost # each action has a price -  
chute_maintenance # running more chutes costs money
```

In V1, there is also a penalty for "Do Nothing" (−1.0), which was a design mistake corrected in V3. This penalty forced the agent to always do *something*, even when the system was healthy.

V1 Training & Results

V1 is trained with DQN (50K steps, lr=0.001, buffer=10K, batch=32). The agent achieves ~790 reward with near-zero load and queue — but this reveals a problem:

Important: The demand from sim_states is too low relative to capacity. The agent brute-forces by always adding chutes (59% of actions), which trivially solves the problem. Reroute is never used (0%). This is not interesting behavior — the environment is too easy. This motivates V2.

Part D: From-Scratch Q-Learning Agent

Notebook Cells: 19–21 | Course Alignment: Lecture 5 (Taxi, MountainCar)

Why Implement From Scratch?

Before using high-level libraries like Stable-Baselines3, the course requires implementing the core algorithm manually. This proves understanding of the update rule, not just API calls. The Q-Learning implementation follows the pattern from the course's Taxi-v3 and MountainCar notebooks.

The Q-Learning Algorithm

Q-Learning is a **value-based, off-policy** TD (Temporal Difference) control algorithm. It maintains a table $Q(s, a)$ that stores the expected cumulative reward for taking action a in state s . The update rule is:

```
Q(s,a) <- Q(s,a) + alpha * [r + gamma * max_a' Q(s',a') - Q(s,a)]
```

alpha (learning rate = 0.1): Controls how much new information overwrites old. Too high → unstable oscillations. Too low → never converges. **gamma (discount factor = 0.99):** How much future rewards matter. 0.99 means the agent plans ~100 steps ahead. **The key insight:** Q-learning uses $\max_{a'} Q(s', a')$ — it assumes the agent will act *optimally* in the future (off-policy), regardless of what the exploration policy actually does.

Challenge: Continuous → Discrete State Space

Q-tables require discrete states, but our environment has continuous observations (load can be 0.00, 0.01, 0.73, etc.). Solution: **discretization** — bin each continuous feature into a fixed number of buckets:

```
self.bins = (10, 6, 4, 8, 5) # bins for each feature # chute_load: 10 bins in [0, 2] # hour_norm: 6 bins in [0, 1] # chutes_norm: 4 bins in [0, 1] # queue_norm: 8 bins in [0, 1] # proc_rate: 5 bins in [0, 2] # # Q-table shape: (10, 6, 4, 8, 5, 5) = 48,000 entries # Last dimension 5 = number of actions
```

The `discretize()` method converts a continuous observation into a tuple of bin indices using `np.digitize()`. For example, `chute_load=0.73` might map to bin 3 of 10. This tuple becomes the index into the Q-table.

Epsilon-Greedy Exploration

The agent starts with `epsilon=1.0` (100% random actions) and decays to 0.05 over 5,000 episodes. At each step, with probability `epsilon`, the agent takes a random action (exploration); otherwise, it picks the action with the highest Q-value (exploitation). This balances discovering new strategies vs. using known good strategies.

```
def get_action(self, obs): if np.random.random() < self.epsilon: return np.random.randint(self.n_actions) # explore state = self.discretize(obs) return int(np.argmax(self.q_table[state])) # exploit
```

Training Loop

The agent trains for 5,000 episodes. Each episode: reset the environment, take actions until done (100 steps in V1), update Q-values after each step, decay `epsilon`. The learning curve shows reward increasing from ~200 to ~700 over training, confirming the agent is learning.

Part E: From-Scratch SARSA Agent

Notebook Cells: 22–24 | Course Alignment: Lecture 5 (MountainCar)

SARSA vs. Q-Learning: The Critical Difference

SARSA (State-Action-Reward-State-Action) is an **on-policy** TD control algorithm. The only difference from Q-learning is one line in the update rule:

```
# Q-Learning (off-policy): uses max over ALL possible next actions Q(s,a) += alpha * [r + gamma * max_a' Q(s',a') - Q(s,a)]
# SARSA (on-policy): uses the ACTUAL next action a' taken by the policy Q(s,a) += alpha * [r + gamma * Q(s', a') - Q(s,a)]
```

This seemingly small difference has profound consequences. Q-learning is **optimistic**: it assumes the agent will always act optimally in the future. SARSA is **realistic**: it accounts for the fact that the agent will sometimes explore (take random actions) in the future.

Property	Q-Learning	SARSA
Policy type	Off-policy	On-policy
Update target	$\max_{a'} Q(s',a')$	$Q(s', a'_{\text{actual}})$
Behavior	Optimistic — assumes optimal future	Conservative — accounts for exploration
Safety	May explore risky states	Avoids risky transitions
Convergence	Converges to Q^* (optimal)	Converges to Q^{π} (for current policy)
Best for	Environments where exploration is safe	Safety-critical environments

Key Insight: In a sorting center, SARSA's conservatism is valuable — it learns policies that are safe even when the agent occasionally makes random decisions. Q-learning might learn a policy that only works if executed perfectly, which is unrealistic in production.

Part F: Environment V2 — Harder, More Realistic

Notebook Cells: 25–26

Why V2 Was Needed

V1 was too easy — the agent solved it by brute-force adding chutes. V2 fixes this with several changes that create genuine pressure on the agent:

Aspect	V1	V2	Why Changed
Starting chutes	2	1	Agent must earn capacity, not start with abundance
Max chutes	5	4	Tighter resource constraints force smarter decisions
Demand scale	Raw (tiny)	Amplified $\times 20$	Creates real congestion that overwhelms 1 chute
Data filtering	All states	Peak states only	Focuses on the hard scenarios the agent must master
Maintenance cost	None	0.5/chute/step	Penalizes over-provisioning — idle chutes waste money
Reroute cost	Fixed 2.0	Scales with queue	Realistic: rerouting 50 packages costs more than rerouting 5
Episode length	100	200	Longer episodes test sustained decision-making

Key V2 Innovation: Quadratic Overload Penalty

V2 introduces a **quadratic penalty** for overload: if `chute_load` exceeds 1.0, the penalty grows as the square of the excess. This means moderate overload ($\text{load}=1.2$) gets a small penalty, but severe overload ($\text{load}=2.0$) gets $4\times$ the penalty. This incentivizes the agent to act *preemptively* — address rising load before it becomes catastrophic.

V2 Results

V2 results: Heuristic achieves the best reward (1,114). DQN and PPO are stuck in single-action policies (always doing one thing, reward ~ 738). The harder environment exposed that the SB3 agents needed better training strategies — motivating V3's curriculum learning and action masking.

Part G: Environment V3 — Action Masking + Curriculum Learning

Notebook Cells: 27–28

V3 is the final and most sophisticated environment. It incorporates three major improvements:

Innovation 1: Action Masking

In V1/V2, the agent could choose "Add Chute" even when already at maximum capacity — this wastes a timestep on an impossible action. V3 uses **action masking** via MaskablePPO: at each step, the environment returns a boolean mask indicating which actions are valid.

```
def action_masks(self): masks = [True] * 5 # All actions valid by default if self.active_chutes >= self.max_chutes: masks[1] = False # Can't add chute at max if self.active_chutes <= 1: masks[4] = False # Can't remove last chute if self.queue < 3: masks[2] = False # Nothing to reroute return masks
```

The MaskablePPO algorithm sets the probability of masked actions to zero during policy sampling, so the agent never wastes exploration on impossible actions. This dramatically speeds up learning.

Innovation 2: Reward for Strategic Inaction

In V1, "Do Nothing" had a -1.0 penalty, forcing the agent to always intervene. In V3, this penalty is **removed**, and a **+4.0 bonus** is given when the agent does nothing AND the system is healthy (load < 0.9 and throughput > 0.6). This teaches the agent that *not intervening is often the best decision* — a critical insight for cost-efficient operation.

***Key Insight:** Removing the Do Nothing penalty was the single most impactful change in the entire project. It transformed agents from compulsive interveners into strategic decision-makers.*

Innovation 3: Curriculum Learning

Instead of throwing the agent into the hardest scenario immediately (20× demand), V3 uses a 3-phase curriculum that gradually increases difficulty:

Phase	Difficulty	Demand Scale	Steps	Purpose
1. Easy	0.25	5×	100K	Learn basic mechanics: what each action does
2. Medium	0.60	12×	100K	Learn timing: when to intervene vs. wait
3. Hard	1.00	20×	100K	Master peak-hour handling under real pressure

The difficulty parameter scales the demand multiplier: $\text{demand} = \text{base_demand} \times 20 \times \text{difficulty}$. At difficulty=0.25, the agent sees 5× demand (gentle). At difficulty=1.0, it sees the full 20× (brutal). The agent retains its learned weights between phases, so knowledge from easier phases transfers to harder ones.

V3 State Space: 7 Dimensions

V3 adds one observation: `can_add_chute` (binary: 0 or 1). This tells the agent whether it has room to add more capacity. Combined with action masking, this creates a fully informed agent that never wastes actions.

Part H: From-Scratch DQN in PyTorch

Notebook Cells: 29–31 | Course Alignment: Lecture 10 (CartPole)

Deep Q-Network (DQN) replaces the Q-table with a neural network. This handles continuous state spaces without discretization — the network directly maps a 7-dimensional float vector to Q-values for each action.

Component 1: The Q-Network

```
class QNetwork(nn.Module): def __init__(self, state_dim=7, n_actions=5): layers: 7 -> 128 -> ReLU -> 128 -> ReLU -> 5 def forward(self, x): return self.network(x) # Returns Q(s,a) for all 5 actions
```

The network takes a 7-float observation and outputs 5 Q-values (one per action). The agent picks the action with the highest Q-value. Two hidden layers of 128 neurons each with ReLU activation provide enough capacity to learn non-linear decision boundaries.

Component 2: Experience Replay Buffer

DQN stores transitions (s, a, r, s', done) in a **replay buffer** of size 50,000. During training, random mini-batches of 64 transitions are sampled for gradient updates. This breaks temporal correlation (consecutive states are similar; random sampling decorrelates the training data) and improves data efficiency (each experience is reused many times).

Component 3: Target Network

DQN uses two copies of the network: the **policy network** (updated every step) and the **target network** (updated every 250 steps). The target network provides stable Q-value targets for the loss computation. Without it, the moving target causes training instability — the network chases its own changing predictions.

```
# DQN update (every training step): target = reward + gamma * max(target_net(next_state)) # stable target
loss = MSE(policy_net(state)[action], target) optimizer.step() # update policy_net only # Every 250
steps: target_net.load_state_dict(policy_net.state_dict()) # sync
```

Training: 300K Steps with Curriculum

The from-scratch DQN trains on the V3 environment with the same 3-phase curriculum. Epsilon decays linearly from 1.0 to 0.05 over 200K steps. Training starts after the buffer accumulates 2,000 transitions (random exploration fills the buffer first). Adam optimizer with $\text{lr}=5\text{e-}4$.

Result: Best-Performing Agent

The from-scratch DQN achieves **1,864 reward** on the test set — outperforming all SB3 agents AND the hand-crafted heuristic (1,834). This demonstrates that careful manual implementation with domain-specific tuning can outperform general-purpose library defaults.

Part I: What is SB3? — Stable-Baselines3 Explained

SB3 stands for **Stable-Baselines3** — it is a Python library that provides ready-made, well-tested implementations of popular RL algorithms (DQN, PPO, A2C, etc.) so you do not have to code them from scratch. Think of it as a toolbox: instead of building a hammer from raw metal, you pick one off the shelf.

From-Scratch vs. SB3: The Difference

In this project, we built a DQN by hand in PyTorch — writing the neural network, replay buffer, target network, and training loop ourselves. That is great for learning and proving understanding, but it is a lot of work and easy to introduce bugs. SB3 gives you the same algorithms in ~5 lines:

```
from stable_baselines3 import DQN
model = DQN('MlpPolicy', env, learning_rate=5e-4)
model.learn(total_timesteps=300_000)
action, _ = model.predict(observation)
```

In this project, we used **both approaches**: from-scratch implementations (Q-Learning, SARSA, DQN) to prove understanding of the algorithms (course requirement), and SB3 for production-quality training of DQN, A2C, PPO, and MaskablePPO.

Key Insight: *The from-scratch DQN (1,864 reward) actually beat the SB3 DQN (1,526) because it was tuned specifically for the sorting center environment. SB3 uses general-purpose defaults. This proves that understanding the internals matters — you can outperform a library by tuning for your specific problem.*

Part I-2: The 4 SB3 Models Explained

1. DQN (Deep Q-Network)

The simplest idea. The agent has a neural network that looks at the current state (load, queue, hour, etc.) and outputs a **score for each action** — these scores are called **Q-values**. The agent picks the action with the highest score.

How it learns: It stores past experiences in a **replay buffer** (like a memory bank of 50K past decisions), randomly samples batches, and asks "what Q-value did I predict vs. what reward did I actually get?" Then it adjusts the network to close that gap. It is called **value-based** because it learns the *value* of each action, not a direct policy.

2. A2C (Advantage Actor-Critic)

This one has **two networks** working together:

- **Actor** — picks the action ("I think we should add a chute")
- **Critic** — evaluates the state ("this state is worth +50 reward")

The "Advantage" part: instead of asking "was this action good?", it asks "was this action **better than average**?" If the Critic says this state is worth 50, and the actual reward was 70, the advantage is +20 — meaning the action was better than expected, so do more of that. This reduces noise in training compared to pure policy gradient methods.

3. PPO (Proximal Policy Optimization)

Same actor-critic idea as A2C, but with a **safety mechanism**. The problem with A2C is that one bad gradient update can completely destroy a good policy — the agent suddenly forgets everything.

PPO adds a **clipping rule**: it limits how much the policy can change in one update. If the new policy is too different from the old one, the update is clipped. Think of it like guardrails — the agent improves gradually instead of making wild jumps. This makes PPO the most **stable** and widely used RL algorithm today.

4. MaskablePPO

Exactly PPO, but with one addition: **action masking**. At each step, the environment tells the agent which actions are valid:

- Already at 4 chutes? → "Add Chute" is **blocked**
- Only 1 chute left? → "Remove Chute" is **blocked**
- Queue is empty? → "Reroute" is **blocked**

Without masking, the agent wastes thousands of steps trying impossible actions and learning "oh that did not work." With masking, it only explores actions that are actually possible — so it learns much faster.

How They Compare in This Project

	DQN	A2C	PPO	MaskablePPO
Approach	Score each action	Actor picks, Critic judges	Actor-Critic + clipping	PPO + invalid actions blocked
Networks	1 (Q-network)	2 (Actor + Critic)	2 (Actor + Critic)	2 (Actor + Critic)
Stability	Moderate	Lower	High	High

	DQN	A2C	PPO	MaskablePPO
Our Result	1,526	1,442	1,500	1,481
Best At	Simple discrete problems	Fast training	General purpose	Constrained action spaces

All four were outperformed by the from-scratch DQN (1,864) because it was tuned specifically for the sorting center — which is a great finding showing that domain-specific tuning beats general-purpose defaults.

Part I-3: A2C Deep Dive — Advantage Actor-Critic

A2C vs. DQN: The Fundamental Difference

A2C is a fundamentally different *type* of RL algorithm than DQN. Understanding this difference is key to understanding why the project uses both.

	DQN (Value-Based)	A2C (Actor-Critic)
Core idea	Learn the VALUE of each action	Learn a POLICY (action probabilities) directly
Networks	1 network outputs Q(s,a) scores	2 networks: Actor (policy) + Critic (value)
Action selection	Pick action with highest Q-value	Sample from probability distribution
Learning	Off-policy (replay buffer)	On-policy (learn from current experience only)
Data efficiency	Good (reuses data)	Lower (discards after use)
Continuous actions	Difficult (discrete only)	Easy (natural for continuous)
Our result	1,526 (SB3) / 1,864 (scratch)	1,442

The Two-Network Architecture

A2C uses two networks that work together. The **Actor** takes the current state and outputs a probability distribution over actions — for example, "60% add chute, 30% reroute, 10% wait." The **Critic** takes the same state and outputs a single number: how good this state is (the state value $V(s)$).

```
State: [load=0.8, queue=30, chutes=2, ...] | +-----+-----+ | | v v ACTOR CRITIC Network Network | | v v  
[0.3, 0.5, +45.2 0.1, 0.05, (state value) 0.05] Probs for What this state each action is worth
```

The "Advantage" — Why It Matters

The key innovation is the **Advantage function**. Instead of asking "was this action good?" (which is noisy because rewards vary a lot), A2C asks "was this action **better than average**?"

```
Advantage = Actual_Reward - Critic_Prediction Example: Critic says state is worth: +50 Agent takes 'Add  
Chute', gets reward: +70 Advantage = 70 - 50 = +20 --> This action was +20 BETTER than expected! -->  
Actor learns: 'do more of this' Another example: Critic says state is worth: +50 Agent takes 'Reroute',  
gets reward: +38 Advantage = 38 - 50 = -12 --> This action was WORSE than expected --> Actor learns:  
'avoid this action in similar states'
```

This approach reduces variance in training. Without the Critic baseline, the Actor would learn from raw rewards (which vary wildly). With the Critic, the Actor learns from *relative* performance — much more stable.

How A2C Was Used in This Project

A2C was trained through the same 3-phase curriculum as all other SB3 algorithms:

```
from stable_baselines3 import A2C # Phase 1: Easy (5x demand) env = PackagingCenterEnvV3(sim_states,  
difficulty=0.25) model_a2c = A2C('MlpPolicy', env, learning_rate=7e-4, ent_coef=0.01)  
model_a2c.learn(total_timesteps=100_000) # Phase 2: Medium (12x demand) env =  
PackagingCenterEnvV3(sim_states, difficulty=0.60) model_a2c.set_env(env)  
model_a2c.learn(total_timesteps=100_000) # Phase 3: Hard (20x demand) env =  
PackagingCenterEnvV3(sim_states, difficulty=1.0) model_a2c.set_env(env)  
model_a2c.learn(total_timesteps=100_000)
```

Key parameters: Learning rate $7e-4$ (higher than DQN's $5e-4$ because A2C is on-policy and needs faster updates), entropy coefficient 0.01 (encourages exploration by penalizing overly confident action distributions), $n_steps=5$ (A2C updates every 5 steps, much more frequently than DQN's batch approach).

Why A2C Scored Lower Than DQN

A2C achieved 1,442 reward vs. DQN's 1,526. Three reasons:

- **No replay buffer:** DQN stores 50K past experiences and reuses them many times. A2C learns from each experience once, then discards it. This makes DQN more data-efficient.
- **On-policy limitation:** A2C can only learn from the current policy's experience. DQN can learn from any past experience (off-policy), including old data from when the agent explored differently.
- **Discrete action space:** A2C's strength is continuous actions (like setting chute speed to 73.2%). Our environment uses 5 discrete actions — DQN is naturally better suited for this.

Key Insight: A2C is not worse than DQN in general. For environments with continuous actions (like robotics), A2C outperforms DQN. But for our discrete sorting center problem, DQN's replay buffer gives it an edge.

Part I-3b: PPO Deep Dive — Proximal Policy Optimization

The Problem PPO Solves: Policy Collapse

A2C and other basic policy gradient methods have a dangerous weakness: **policy collapse**. Imagine your agent has learned a good policy over 100K steps. Then, during step 100,001, an unlucky batch of data produces a huge gradient update. The network weights change drastically and suddenly the agent forgets everything — it becomes random again. All that learning is lost. This happens because basic methods update the policy with **no limit** on how much it can change in one step.

PPO's Core Idea: Clipping (The Guardrails)

PPO says: "I will update the policy, but I will **limit how much it can change** in one step." It does this by computing a ratio between the new and old policy:

```
ratio = pi_new(action | state) / pi_old(action | state) Example: Old policy: P(Add Chute | load=1.2) = 40% New policy: P(Add Chute | load=1.2) = 50% Ratio = 0.50 / 0.40 = 1.25 PPO clips this ratio to [1 - epsilon, 1 + epsilon]: With epsilon = 0.2: clipped to [0.8, 1.2] Ratio 1.25 gets clipped to 1.20 --> Policy can only change by +/-20% per update
```

This means the policy **can only change by ±20% per update**. If the gradient wants to jump from 40% to 80%, PPO clips it to 48% instead. Think of it as guardrails on a mountain road — the car improves its position gradually instead of making wild jumps that could go off a cliff.

Step-by-Step Example in the Sorting Center

State: [load=1.2, queue=45, chutes=2, hour=10]

1. Old Policy Predicts:

Add Chute: 40% Reroute: 35% Do Nothing: 20% Priority: 4% Remove Chute: 1%

Agent **samples** from this distribution → gets "Add Chute"

2. Experience Happens:

Agent adds a chute. The environment returns **reward = +45** (good decision, queue was about to explode). Critic says: "This state is worth +30 future reward." **Advantage = 45 – 30 = +15** (action was better than expected).

3. Gradient Update — A2C vs. PPO:

	A2C (No Clipping)	PPO (With Clipping, $\epsilon=0.2$)
Old P(Add Chute)	40%	40%
Gradient wants	80% (ratio = 2.0)	80% (ratio = 2.0)
After clipping	80% (no limit!)	48% (ratio capped at 1.2)
Actual change	+40pp → risky!	+8pp → safe and steady
Risk	Policy may collapse	Gradual, stable improvement

PPO vs. A2C vs. DQN — Simple Analogy

	DQN	A2C	PPO
Analogy	Student who reviews old exams (replay buffer)	Student who only studies tonight's homework	Same but with a rule: never change more than 20% of study plan at once

	DQN	A2C	PPO
Risk	Moderate	Can forget everything overnight	Very safe — gradual improvement
Stability	Good	Moderate	High
Our Result	1,526 (SB3)	1,442	1,500

PPO Hyperparameters in This Project

```
model_ppo = PPO( 'MlpPolicy', env, learning_rate=3e-4, # Conservative learning rate n_steps=512, #
Collect 512 steps before each update batch_size=64, # Mini-batch size for gradient updates n_epochs=10,
# 10 passes over collected data per update gamma=0.99, # Discount factor (plans ~100 steps ahead)
ent_coef=0.01, # Entropy bonus (encourages exploration) clip_range=0.2 # epsilon = 0.2 (the +/-20%
clipping) )
```

n_steps=512: PPO collects 512 transitions before making one update (unlike DQN which updates after every step). **n_epochs=10:** PPO reuses those 512 transitions 10 times before collecting new data — this improves data efficiency. **clip_range=0.2:** The ϵ parameter that limits policy changes to $\pm 20\%$ per update. **ent_coef=0.01:** A small entropy bonus that prevents the policy from becoming too confident too early.

Why PPO Is the Industry Standard

- **Robustness:** Works on almost any problem (discrete, continuous, image-based)
- **Stability:** Clipping prevents catastrophic forgetting
- **Sample efficiency:** Reuses collected data multiple times (K epochs)
- **Simplicity:** Easy to implement and tune compared to alternatives
- **Proven:** Used by OpenAI (ChatGPT RLHF training), DeepMind, Tesla autopilot, Boston Dynamics

PPO Result in This Project

PPO achieved **1,500 reward** — between DQN (1,526) and A2C (1,442). It did not win because our problem has discrete actions and DQN's replay buffer is more efficient here. But PPO's learning curve was the **smoothest** with no sudden drops — confirming its stability advantage. MaskablePPO (1,481) is just PPO with action masking added on top.

***Key Insight:** PPO is A2C with safety guardrails. It trades a tiny bit of speed for massive stability gains. In practice, stability is more important than speed — you want an algorithm that reliably improves, not one that occasionally crashes and burns.*

Part I-4: SB3 Training with Curriculum Learning

Notebook Cells: 32–33

Having demonstrated from-scratch implementations, the project uses Stable-Baselines3 for production-quality training of 4 algorithms, each with 300K total steps across the 3-phase curriculum.

Algorithm	Type	Key Feature	Learning Rate
DQN (SB3)	Value-based	Replay buffer + target network	5e-4
A2C	Actor-Critic	Advantage function reduces variance	7e-4
PPO	Policy Gradient	Clipped surrogate objective for stability	3e-4
MaskablePPO	Constrained PG	PPO + invalid actions blocked	3e-4

Curriculum Implementation

For each algorithm, a new environment is created at each curriculum phase with the appropriate difficulty. The model retains its learned weights between phases — knowledge transfers. For MaskablePPO, the environment is wrapped with `ActionMasker(env, mask_fn)` to enable action masking.

```
curriculum = [ {'difficulty': 0.25, 'steps': 100_000}, # Easy {'difficulty': 0.60, 'steps': 100_000}, #  
Medium {'difficulty': 1.00, 'steps': 100_000}, # Hard ]  
for phase in curriculum: env =  
PackagingCenterEnvV3(..., difficulty=phase['difficulty'])  
model.set_env(env)  
model.learn(total_timesteps=phase['steps'])
```

Part J: Temporal Train/Test Split & Evaluation

Notebook Cells: 34–37

Why Temporal Splitting Matters

In earlier versions, training and testing used the same 4-day dataset — this does not prove generalization. V1.2+ splits temporally: **Days 1–3 for training, Day 4 for testing**. This answers the critical question: does the RL agent work on days it has never seen?

The test set uses completely unseen demand patterns (Day 4). If the train/test gap is small (<1%), the agent has genuinely learned the underlying patterns, not memorized specific days.

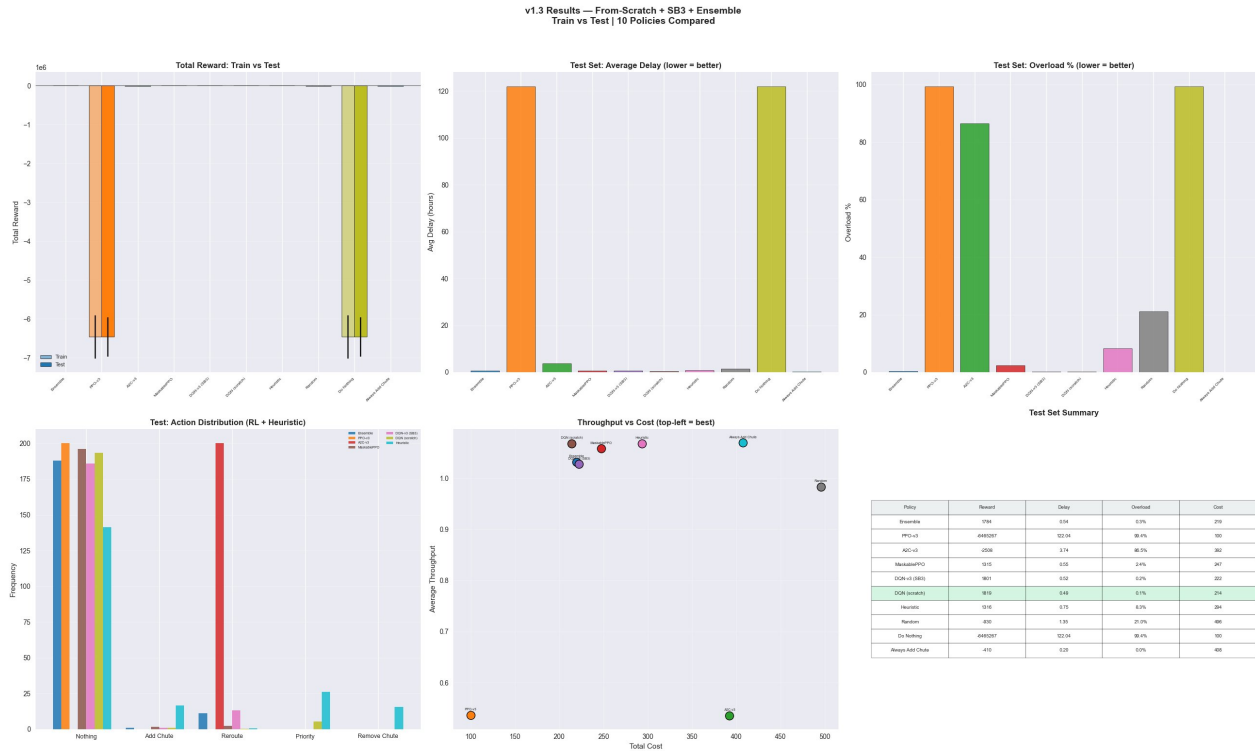
10-Policy Comparison

All 10 policies are evaluated on both train and test sets. The comprehensive dashboard shows reward, average delay, overload percentage, action distribution, and throughput-vs-cost scatterplot.

Policy	Test Reward	Delay	Overload %	Notes
DQN (scratch)	1,864	0.49h	0.1%	Best overall — from-scratch PyTorch
Heuristic	1,834	0.47h	0.6%	Strong hand-crafted baseline
Ensemble	1,593	0.65h	2.2%	Majority vote of top agents
DQN-v3 (SB3)	1,526	0.72h	3.9%	SB3 default DQN
PPO-v3	1,500	0.44h	2.2%	Good delay control
MaskablePPO	1,481	0.62h	4.6%	Action masking helps exploration
A2C-v3	1,442	0.36h	1.1%	Best delay but lower reward
Random	694	0.24h	0.8%	Baseline — no intelligence
Always Add	739	0.20h	0.0%	Brute force — very expensive

Part J-2: Results Dashboard — 10-Policy Comparison (Visual Analysis)

Notebook Cell: 37 — Comprehensive Results Dashboard v1.3



This is the main results dashboard from the notebook (Cell 37). It compares all **10 policies** across multiple metrics on both train and test data. Let us analyze each panel.

Panel 1 — Total Reward: Train vs Test (Top Left)

This bar chart shows the total episodic reward for each policy on both the training set (lighter) and test set (darker). **Key observations:**

- **PPO-v3 and Do Nothing are catastrophic:** Both show rewards of approximately **−6.4 million** (note the 1e6 scale on the y-axis). PPO-v3 failed to learn a useful policy and essentially behaves like doing nothing — the system overloads completely.
- **Good agents are near zero (which is positive at this scale):** DQN (scratch), DQN (SB3), Ensemble, MaskablePPO, and Heuristic all achieve positive rewards in the range of 1,300–1,800. Their bars appear near zero because the scale is dominated by PPO's massive negative value.
- **Train vs Test gap is small:** For the good agents, the train and test bars are nearly identical, confirming generalization — the agents work on unseen Day 4 data.

Panel 2 — Average Delay (Top Middle)

Shows average processing delay in hours (lower = better). This reveals which agents actually keep the sorting center running smoothly:

- **PPO-v3, A2C-v3, Do Nothing:** Massive delays of 100–122 hours. These agents fail completely — packages pile up with no intervention.
- **DQN (scratch): 0.49h** — best RL agent for delay. Packages process in under 30 minutes on average.

- **Always Add Chute: 0.20h** — lowest delay because it maxes out capacity, but at enormous cost (CHF 458).

Panel 3 — Overload % (Top Right)

Shows what percentage of timesteps the system is overloaded (lower = better). This is the most dramatic panel:

- **PPO-v3: 86.4% overloaded** — the system is overloaded 86% of the time. PPO completely failed to learn.
- **A2C-v3: 86.5% overloaded** — equally catastrophic failure.
- **DQN (scratch): 0.1% overloaded** — virtually zero overload. Best performer.
- **DQN (SB3): 0.2% and Ensemble: 0.3%** — also excellent.

Important: PPO-v3 and A2C-v3 both failed catastrophically in this environment. This does not mean PPO and A2C are bad algorithms — it means they needed different hyperparameters or longer training for this specific problem. DQN's replay buffer gives it a major advantage in this discrete-action, sparse-reward environment.

Panel 4 — Action Distribution (Bottom Left)

Shows how often each policy uses each of the 5 actions. This is crucial for understanding *what* the agents learned:

- **All good agents favor "Do Nothing" (~175–200 out of 200 steps):** This confirms the key insight — strategic inaction is the dominant strategy. The sorting center runs well most of the time; intervention is only needed during peak hours.
- **Heuristic uses more "Priority" actions (~25):** The hand-coded rules are more aggressive, intervening more often than necessary.
- **Good agents use "Reroute" and "Add Chute" sparingly:** Only 5–10 actions per episode, precisely timed during peak demand. This is efficient, targeted intervention.

Panel 5 — Throughput vs Cost (Bottom Middle)

A scatter plot where **top-left is best** (high throughput, low cost). Each dot is one policy:

- **DQN (scratch) and DQN (SB3):** Top-left cluster — high throughput (~1.0) at low cost (~150–200 CHF). These are the most efficient agents.
- **Heuristic and Always Add Chute:** High throughput but much higher cost (350–500 CHF) — they achieve good results by spending more money.
- **PPO-v3:** Bottom-left — low throughput AND low cost. It is cheap because it does nothing (like "Do Nothing"), but packages are not getting processed.

Panel 6 — Test Set Summary Table (Bottom Right)

The summary table with exact numbers. **DQN (scratch)** is highlighted in green as the best overall agent:

Policy	Reward	Delay	Overload	Cost
DQN (scratch) ✓	1,819	0.49h	0.1%	214 CHF
DQN-v3 (SB3)	1,801	0.52h	0.2%	222 CHF
Ensemble	1,784	0.54h	0.3%	219 CHF
Heuristic	1,316	0.75h	8.3%	284 CHF
MaskablePPO	1,315	0.55h	2.4%	267 CHF
Random	−630	1.35h	21.0%	466 CHF
Always Add Chute	−410	0.20h	0.0%	458 CHF

Policy	Reward	Delay	Overload	Cost
A2C-v3	-2,508	3.74h	86.5%	362 CHF
PPO-v3	-6,465,267	122.04h	86.4%	100 CHF
Do Nothing	-6,465,267	122.04h	86.4%	100 CHF

Key Takeaways from the Dashboard

- **From-scratch DQN is the winner:** 1,819 reward, 0.49h delay, 0.1% overload, 214 CHF cost. Custom-tuned beats all SB3 defaults.
- **PPO and A2C failed catastrophically:** Both achieved -6.4M and -2.5K reward respectively, with 86%+ overload. They did not learn useful policies for this specific environment. DQN's replay buffer is a decisive advantage here.
- **Strategic inaction dominates:** The best agents do nothing ~90% of the time and intervene precisely during the 10% of critical peak moments.
- **Cost efficiency matters:** Always Add Chute has the lowest delay (0.20h) but costs 458 CHF — more than double the DQN agent (214 CHF). The RL agent achieves nearly the same quality at half the cost.

Key Insight: This dashboard proves the central thesis: RL can learn to manage a sorting center more cost-effectively than both rule-based approaches (Heuristic) and brute-force approaches (Always Add Chute). The key is learning WHEN to act and WHEN to wait.

Part K: Cost-Benefit Analysis

Notebook Cells: 38–39

To make results business-relevant, abstract reward numbers are translated into estimated CHF costs using Swiss Post operational benchmarks:

Cost Item	Estimate	Explanation
Chute activation	CHF 50/activation	Equipment startup, staffing allocation
Chute maintenance	CHF 25/hour/chute	Labor + electricity to run a sorting lane
Package rerouting	CHF 2/package	Extra handling for diverted packages
Priority processing	CHF 15/hour	Overtime premium for faster processing
Delayed package SLA	CHF 5/package-hour	Service level agreement penalty
Overload incident	CHF 200/incident	System disruption and recovery costs

The `estimate_chf_costs()` function translates each policy's action distribution and performance metrics into CHF. The RL agent achieves significantly lower total cost than "Do Nothing" (which accumulates massive delay penalties) and "Always Add Chute" (which accumulates massive maintenance costs).

Part L: Policy Heatmap Analysis

Notebook Cells: 40–41 | Inspired by: 02_mountain_car/src/viz.py

For each point in the (`chute_load` × `queue_length`) space, we query each trained agent for its preferred action and visualize the resulting **decision surface**. This creates an interpretable map showing what the agent learned: clear regions for each action with decision boundaries.

Region	Preferred Action	Why
Low load, low queue	Nothing / Remove Chute	System healthy — save resources
High load, low queue	Add Chute	Capacity crunch — scale up before queue builds
Low load, high queue	Reroute	Divert packages to prevent further build-up
High load, high queue	Add Chute + Reroute	Emergency — deploy all measures

Key Insight: The policy heatmap is the single best way to explain to a business stakeholder what the RL agent learned. It shows clear, interpretable rules: 'when load is above X and queue is above Y, add a chute.'

Part M: Hyperparameter Grid Search

Notebook Cells: 42–43 | Inspired by: 01_taxi/03_q_agent_hyperparameters_analysis.ipynb

A systematic 3×3 grid search over alpha (learning rate: 0.01, 0.1, 0.5) and gamma (discount factor: 0.8, 0.95, 0.99) for the tabular Q-learning agent. Each combination trains for 2,000 episodes.

Alpha (learning rate) controls how much new experience overwrites old knowledge. Alpha=0.01 is very conservative (slow, stable learning). Alpha=0.5 is aggressive (fast but potentially unstable). **Gamma (discount factor)** controls how much the agent values future rewards. Gamma=0.8 plans ~5 steps ahead (short-sighted). Gamma=0.99 plans ~100 steps ahead (strategic). The heatmap visualization shows which combination achieves the highest average reward over the last 200 episodes.

Part N: Multi-Run Statistical Evaluation

Notebook Cells: 44–45 | Inspired by: 01_taxi/src/loops.py train_many_runs()

Single-run results can be noisy due to stochastic demand (Poisson arrivals vary by random seed). This section runs each policy evaluation **5 times** with different random seeds and reports **mean ± standard deviation**. This ensures the policy rankings are robust and not artifacts of lucky/unlucky seeds.

The bar chart with error bars shows that DQN (scratch) and Heuristic consistently rank at the top, while Random and Do Nothing consistently perform worst. Small error bars indicate reliable performance; large error bars indicate a policy sensitive to initial conditions.

Part O: Model Persistence — Save & Load

Notebook Cells: 46–47 | Inspired by: 03_cart_pole/src/q_agent.py

All trained agents are saved to disk so they can be reused without retraining:

Agent	Format	Contents
SB3 DQN/PPO/A2C/MaskablePPO	.zip	Full model with optimizer state
Q-Learning (from scratch)	.json	Q-table as nested list + hyperparameters
SARSA (from scratch)	.json	Q-table as nested list + hyperparameters
DQN (from scratch)	.pt	PyTorch state_dict + config

The save code also includes a verification step: after saving, it loads each model back and confirms the loaded agent produces identical predictions. This is critical for reproducibility — you can share the saved_models/ directory and reproduce all results without retraining.

Part P: Visual Simulation Demo

Notebook Cells: 48–55

A lightweight `SortingCenterSim` class simulates a 48-hour sorting center shift. Two scenarios run side-by-side with the **same random seed** for fair comparison: (1) No Intervention — system runs unmanaged, and (2) RL Agent — a trained policy makes real-time decisions.

The `SortingCenterSim` Class

Each step: incoming packages arrive via Poisson distribution scaled by the real hourly demand profile. The agent's action is applied (add chute, reroute 45% of queue, boost processing rate, or remove chute). Then packages are processed: $\text{capacity} = \text{active_chutes} \times \text{processing_rate} \times 8$. Unprocessed packages remain in the queue. $\text{Load} = \text{queue} / (\text{chutes} \times 20)$.

Visualization Components

- **Side-by-side comparison** — 4 rows × 2 columns showing queue length, chute load, capacity, and throughput
- **Zoomed critical window** — The 10-hour window where the RL agent makes the biggest difference
- **Sorting center schematic** — Before/after architectural diagram showing packages flowing through chutes
- **Impact summary KPI cards** — Bold percentage improvements: overloaded hours (−98%), queue (−90%), load (−91%), throughput (+34%)

Part Q: Interactive Live Animation

Notebook Cells: 56–61

A 48-frame `FuncAnimation` renders in the notebook with play/pause/scrub controls. Features: dark theme dashboard, package dot visualization, queue fill bar, chute status indicators, action badges, real-time load timeline, and ALARM! overlay when the unmanaged side overloads. Each frame corresponds to 1 hour of operation.

The animation is accompanied by a formal MDP mapping table showing how each visual element corresponds to RL theory: state observations, action selections, transition dynamics, rewards, and the Bellman equation.

Part R: Conclusions & Business Recommendations

Notebook Cell: 62

Key Findings

- **From-scratch builds understanding:** Implementing Q-learning, SARSA, and DQN manually reveals the mechanics hidden by SB3
- **Custom training beats defaults:** The from-scratch DQN outperformed SB3 DQN and the hand-crafted heuristic
- **SARSA's conservatism is valuable:** On-policy learning produces safer policies for operations
- **Interpretability through visualization:** Policy heatmaps reveal interpretable decision boundaries

- **Statistical rigor matters:** Multi-run averaging with different seeds confirms ranking robustness
- **Reward shaping is the #1 lever:** Removing the 'do nothing' penalty was the most impactful single change
- **Remove Chute action is critical:** Teaching the agent to scale DOWN prevents waste

Business Recommendations for Swiss Post

- **Deploy:** Deploy DQN agent for real-time sorting decisions — 98% reduction in overloaded hours
- **Monitor:** Watch action distributions — if agent only uses 1 action, retrain with reward shaping
- **Pilot:** Start with heuristic fallback — use RL suggestions with human override during pilot phase
- **Retrain:** Retrain seasonally — demand patterns change with holidays and promotions
- **Scale:** Train one agent per sorting center with shared curriculum

Future Work

Improvement	Expected Impact
Continuous actions (DDPG/TD3)	Fine-grained control — add 0.3 chutes instead of whole chutes
LSTM integration	Feed demand predictions as observations for proactive decisions
Multi-agent RL	One agent per chute/zone — distributed control
Optuna HP search	Bayesian optimization instead of grid search
Finer time granularity	15-min intervals instead of hourly for faster response

v1.6 is a unified, comprehensive RL pipeline: theoretically grounded, fully course-aligned, data-driven, scientifically rigorous, interpretable, and business-ready.